

Drydock — Talking-Points Outline

Compressed cue cards. One block per slide. Use if you'd rather speak extemporaneously than read the script.

Act 1 — The setup (0:00–2:15)

1. **Title** — Build software from specs *with a real process* so you can reproduce it. Speed was never the issue.
2. **The gap** — SDD today ships specs but threw away 25 years of software best practices. *To a financial engineer: a number — or a build — you can't reproduce isn't a result.* No process, no reproducible build. Bottleneck = missing process, not the model.
3. **Insight** — Reproducible builds need a process, and it already exists: **Agile**. The Agile Manifesto turns 25 this year; how devs ship at major companies. Drydock = Agile applied to AI delivery. About system *behavior*, not sub-agents. Spec = single source of truth.

Act 2 — The product (2:15–3:50)

4. **Drydock + SAIL** — Governed, Blueprint-driven CLI. SAIL is an *agile process*: **Set up, Analyze, Implement, Loop**. Notes/specs → feature+story files → dependency graph → working software → kept in sync, documented, scored.
5. **The Commander** — Your role changed: not developer/designer → **Commander** (the Agile product owner). Commander decides; the LLM **Agile Best Practices Team** develops; the QuarterDeck is how you talk to them. Behavior, not sub-agents. Compass = intent, Ship's Log = decisions, QuarterDeck = your agile interface.

Act 3 — Walk the phases (3:50–9:00)

6. **S / Set up** — `pipx install, config, init`. Runs on Claude/Codex subscription. No API keys. Just a subscription and an idea.
7. **A / Analyze** — Import md/source/Spec Kit/notes → `analyze` decomposes into stories, milestones, blockers, questions. *Blocker* → *stops and asks*. Output = a *tentative* set of Features, Stories, Spikes; nothing committed until you approve.
8. **QuarterDeck** (*core differentiator*) — Your **Agile web interface** to the process: an optimized path between Commander and the LLM team. Renders the story hierarchy, blockers, questionnaires; approve/answer/redirect → persisted → carried forward. `rigging compact` gives minimal context with Builder/User views. *Instead of guessing, the path 25 years of agile already worked out.* Commander controls → goal is working software you can iterate.
9. **Manifest** (*engine*) — `plan` → typed Blueprints + dependency graph. LLM estimates **Story Points (token cost)** per story against the graph. Drydock stacks the minimal correct files into each build prompt. *Context engineered, not hoped for.*
10. **I / Implement** — `build` walks the graph, builds the runnable frontier one context-optimized step at a time; evidence per step; verify against acceptance → unlock dependents. `build score` = 7-dimension delivery health (completeness, tests, drift...).
11. **Rigging** — Branding + stack rules injected everywhere. Builder gets full spec; consumer gets compacted how-to. Keeps context lean at scale.

12. **L / Loop** — **refit**, two ways: (1) edit spec files → tracks git commit per file → rebuilds only what changed; (2) change tickets against existing specs, built with minimized context. Spec stays source of truth. Minimal drift.

Act 4 — The close (9:00–10:00)

13. **Why different** — Vibe = no spec, no process. SDD = spec, no process. **Drydock = spec + a real Agile process**. Commander decides, team develops, every decision logged. Goal = working software you can reliably iterate, with an easy path to rebuild anytime.
14. **CTA** — **pipx install drydock-sdd**. Open spec, open methodology, recruiting Commanders. Link.

Anticipated Q&A (for the description / pinned comment)

- **vs. GitHub Spec Kit?** Spec Kit authors specs; Drydock adds the missing *process* — an Agile delivery loop with a Commander (product-owner) role, the QuarterDeck review interface, a dependency-graph Manifest, context-budgeted build stacking, and drift scoring/refit. It governs the *whole* loop and can import Spec Kit projects.
- **Which models?** Any subscription-authenticated CLI agent — Claude or Codex today. No raw API keys.
- **Does it lock me in?** Outputs are plain Markdown Blueprints + standard source in your own git repo. Uninstalling removes the CLI, not your work.
- **Determinism?** The scoring/state math is deterministic and lives in the command; the LLM judges and drafts, the command computes and writes.